

ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ИРКУТСКОЙ ОБЛАСТИ  
«АНГАРСКИЙ ТЕХНИКУМ СТРОИТЕЛЬНЫХ ТЕХНОЛОГИЙ»

Курсовая работа

Тема: Разработка интернет-магазина цветочного салона

Разработал \_\_\_\_\_ Власова М.М

Руководитель \_\_\_\_\_ Денисюк А.В

Ангарск, 2026



# СОДЕРЖАНИЕ

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                                                | 6  |
| 1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....                                               | 9  |
| 1.1 Общая характеристика предметной области.....                             | 9  |
| 1.2 Базовые технологии создания сайта.....                                   | 10 |
| 1.3 Использование редактора кода.....                                        | 11 |
| 1.3.1 редактор кода Visual Studio Code .....                                 | 11 |
| 1.3.2 Клиент-серверная архитектура веб-приложения .....                      | 13 |
| 1.4 Выбор технологического стека.....                                        | 14 |
| 1.4.1 Фреймворк Express в серверной части.....                               | 14 |
| 1.4.2 EJS-шаблоны.....                                                       | 15 |
| 1.4.3 Использование базы данных MySQL.....                                   | 17 |
| 1.4.4 Хеширование паролей с использованием bcryptjs.....                     | 18 |
| 1.4.5 Загрузка изображений товаров с использованием Multer.....              | 19 |
| 1.4.6 Связь используемых технологий с функционалом интернет-магазина.....    | 21 |
| 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ ИНТЕРНЕТ-МАГАЗИНА ЦВЕТОВ.....      | 23 |
| 2.1. Назначение и функциональные возможности разработанного приложения ..... | 23 |
| 2.2 Обоснование применения выбранного технологического стека .....           | 24 |
| 2.2.1 Серверная платформа Node.js.....                                       | 24 |
| 2.3. Обоснование применения выбранного технологического стека .....          | 25 |
| 2.3.1. Фреймворк Express.....                                                | 25 |
| 2.3.4. Шаблонизатор EJS .....                                                | 26 |
| 2.3.5. База данных MySQL.....                                                | 27 |
| 2.2.6. Дополнительные библиотеки.....                                        | 28 |
| 2.4. Архитектура и структура проекта .....                                   | 29 |
| 2.4.1. Общая архитектурная модель .....                                      | 29 |
| 2.4.2. Маршруты приложения .....                                             | 29 |
| 2.4.3. Контроллеры как центр бизнес-логики.....                              | 31 |
| 2.4.4. Middleware в проекте .....                                            | 32 |

|         |        |             |        |         |      |                                                |  |               |        |
|---------|--------|-------------|--------|---------|------|------------------------------------------------|--|---------------|--------|
|         |        |             |        |         |      | КР-09.02.07-В-63-26ПЗ                          |  |               |        |
| Изм.    | Кол.уч | Лист        | № док. | Подпись | Дата |                                                |  |               |        |
| Разраб. |        | Власова М.М |        |         |      | Разработка интернет-магазина цветочного салона |  | <u>Лист</u>   | Листов |
| Пров.   |        | Денисюк А.В |        |         |      |                                                |  | 3             | 52     |
|         |        |             |        |         |      |                                                |  | ГАПОУ ИО АТСТ |        |
| Н.контр |        |             |        |         |      |                                                |  |               |        |
| Утв.    |        |             |        |         |      |                                                |  |               |        |



|                                                |    |
|------------------------------------------------|----|
| 2.12.2. Шапка сайта и навигация .....          | 44 |
| 2.12.3. Главная страница.....                  | 44 |
| 2.12.4. Каталог и карточки товаров .....       | 44 |
| 2.12.5. Формы.....                             | 44 |
| 2.12.6. Flash-сообщения .....                  | 45 |
| 2.13. Обработка ошибок и валидация данных..... | 45 |
| 2.13.1. Валидация входных данных.....          | 45 |
| 2.13.2. Обработка ошибок.....                  | 45 |
| 2.14. Результаты работы программы.....         | 46 |
| 2.14.1. Практический результат реализации..... | 46 |
| 2.14.2. Значение реализованных механизмов..... | 47 |
| ЗАКЛЮЧЕНИЕ .....                               | 49 |
| ПРИЛОЖЕНИЕ А .....                             |    |
| ПРИЛОЖЕНИЕ Б.....                              |    |
| ПРИЛОЖЕНИЕ В .....                             |    |
| ПРИЛОЖЕНИЕ Г.....                              |    |
| ПРИЛОЖЕНИЕ Д .....                             |    |

|         |             |      |        |         |      |                                                |               |        |
|---------|-------------|------|--------|---------|------|------------------------------------------------|---------------|--------|
|         |             |      |        |         |      | КР-09.02.07-В-63-26ПЗ                          |               |        |
| Изм.    | Кол.уч      | Лист | № док. | Подпись | Дата |                                                |               |        |
| Разраб. | Власова М.М |      |        |         |      | Разработка интернет-магазина цветочного салона | <u>Лист</u>   | Листов |
| Пров.   | Денисюк А.В |      |        |         |      |                                                |               | 5      |
|         |             |      |        |         |      |                                                |               | 52     |
| Н.контр |             |      |        |         |      |                                                | ГАПОУ ИО АТСТ |        |
| Утв.    |             |      |        |         |      |                                                |               |        |

## ВВЕДЕНИЕ

В настоящее время интернет-торговля занимает значительное место в сфере розничных продаж. Пользователи всё чаще предпочитают оформлять заказы через веб-сайты, так как это позволяет быстро ознакомиться с ассортиментом, выбрать подходящий товар, указать необходимые данные и оформить покупку без посещения обычного магазина. Особенно актуальным такой подход является для цветочных магазинов, где покупателю важно заранее увидеть внешний вид букета, узнать его стоимость, состав, наличие и возможность оформления доставки. Интернет-магазин цветов представляет собой веб-приложение, предназначенное для онлайн-продажи букетов и цветочных композиций. В отличие от простого информационного сайта, такая система должна обеспечивать не только просмотр ассортимента, но и полноценную обработку пользовательских действий: регистрацию и авторизацию, добавление товаров в корзину, оформление заказа, хранение данных в базе, загрузку изображений товаров и управление каталогом через административную панель. Актуальность темы курсовой работы связана с тем, что цветочный бизнес активно использует интернет-продажи. Покупателям удобно выбирать букеты онлайн, сравнивать варианты по цене и внешнему виду, а также оформлять доставку к определённом адресу. Для владельца магазина веб-приложение позволяет систематизировать ассортимент, быстрее обновлять информацию о товарах, управлять заказами и взаимодействовать с клиентами через единую информационную систему. Разработка интернет-магазина цветов также имеет учебную значимость, так как позволяет на практике рассмотреть основные принципы веб-разработки. В ходе создания такого проекта используются базовые технологии клиентской части — HTML, CSS и JavaScript, серверный фреймворк Express, шаблонизатор EJS, база данных MySQL, загрузка файлов с помощью Multer и хеширование паролей через bcryptjs. Эти технологии применяются не

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 6    |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

абстрактно, а для реализации конкретных функций готового приложения. Целью данной курсовой работы является разработка веб-приложения «Интернет-магазин цветов», предназначенного для просмотра цветочной продукции, работы с корзиной, оформления заказов и управления ассортиментом через административную панель. Для достижения поставленной цели необходимо решить следующие задачи:

- Изучить предметную область онлайн-продажи цветов и определить основные функции разрабатываемого приложения.
- Рассмотреть базовые технологии создания веб-сайтов: HTML, CSS и JavaScript.
- Описать использование редактора кода Visual Studio Code при разработке проекта.
- Изучить принципы клиент-серверной архитектуры веб-приложения.
- Обосновать применение фреймворка Express для серверной части интернет-магазина.
- Рассмотреть использование EJS-шаблонов для формирования страниц сайта.
- Описать применение базы данных MySQL для хранения информации о пользователях, товарах, корзине и заказах.
- Рассмотреть загрузку изображений товаров с использованием библиотеки Multer.
- Описать применение bcryptjs для безопасного хранения паролей пользователей.
- Реализовать пользовательскую часть интернет-магазина: каталог, карточку товара, корзину и оформление заказа.
- Реализовать административную часть для управления товарами и заказами.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 7    |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

– Проверить работоспособность основных сценариев веб-приложения.

Объектом исследования является процесс разработки веб-приложения для электронной торговли. Предметом исследования являются технологии и программные средства, применяемые при создании интернет-магазина цветов на основе Node.js, Express, EJS и MySQL. Практическая значимость работы заключается в создании функционирующего веб-приложения, которое может быть использовано как учебный пример интернет-магазина. Разработанный проект демонстрирует основные этапы создания серверного сайта: проектирование структуры приложения, работу с базой данных, создание пользовательского интерфейса, обработку заказов, загрузку изображений и реализацию механизмов авторизации.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 8    |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |



# 1. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

## 1.1 Общая характеристика предметной области

Интернет-магазин цветов представляет собой веб-приложение, предназначенное для демонстрации цветочной продукции, оформления заказов и управления ассортиментом через административную часть. В отличие от обычного информационного сайта, такой проект должен выполнять не только функцию представления информации, но и обеспечивать обработку пользовательских действий: просмотр каталога, выбор товара, добавление позиции в корзину, оформление заказа, регистрацию и авторизацию пользователей, а также добавление и изменение товаров администратором. Предметная область цветочного магазина имеет ряд особенностей. Основной товар представлен букетами, цветочными композициями и отдельными видами цветов. Для пользователя важно видеть изображение товара, его название, стоимость, описание и доступность для заказа. Визуальная часть играет особенно важную роль, так как покупатель принимает решение в основном по внешнему виду букета. Поэтому в системе должна быть реализована возможность хранения и отображения изображений товаров. Интернет-магазин цветов должен быть удобен как для покупателя, так и для администратора. Покупатель взаимодействует с пользовательской частью сайта: просматривает каталог, открывает карточку товара, добавляет выбранные позиции в корзину и оформляет заказ. Администратор работает с внутренней частью приложения: добавляет новые букеты, редактирует сведения о товарах, загружает изображения и просматривает поступившие заказы. Разрабатываемое приложение строится как серверное веб-приложение. Это означает, что основная логика обработки данных находится на сервере. Браузер пользователя отправляет запросы, сервер обрабатывает их, обращается к базе данных и возвращает сформированную страницу. Такой подход подходит для интернет-магазина цветов, так как он позволяет централизованно управлять товарами,

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 9    |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

пользователями и заказами. В проекте используются технологии HTML, CSS и JavaScript для создания пользовательского интерфейса, Express для серверной части, EJS для формирования страниц, MySQL для хранения данных, Multer для загрузки изображений товаров и bcryptjs для безопасного хранения паролей пользователей. Все эти технологии применяются не отдельно друг от друга, а в составе единой архитектуры интернет-магазина.

## 1.2 Базовые технологии создания сайта

Создание интернет-магазина цветов начинается с базовых технологий веб-разработки. К ним относятся HTML, CSS и JavaScript. Именно они обеспечивают отображение страниц в браузере и взаимодействие пользователя с интерфейсом. HTML используется для построения структуры веб-страниц. С его помощью создаются основные элементы сайта: шапка, навигационное меню, каталог товаров, карточки букетов, формы регистрации и входа, корзина, форма оформления заказа и страницы административной панели. HTML задаёт логическую основу страницы и определяет, какие элементы будут доступны пользователю. В интернет-магазине цветов HTML-разметка применяется для вывода информации о товарах. Например, карточка букета может содержать изображение, название, цену и кнопку перехода к подробной информации. На странице товара HTML используется для размещения описания, изображения и формы добавления в корзину. В административной панели HTML-формы применяются для добавления и редактирования товаров. CSS отвечает за внешний вид сайта. В цветочном магазине оформление особенно важно, так как визуальное восприятие напрямую влияет на впечатление пользователя. С помощью CSS задаются цвета, размеры блоков, отступы, расположение карточек, оформление кнопок, форм и навигации. Благодаря стилям сайт выглядит целостно и удобно воспринимается пользователем. Для каталога цветов CSS позволяет оформить товары в виде аккуратной сетки карточек. Каждая карточка может иметь изображение букета, название, цену и кнопку действия. Также стили

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 10   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

используются для оформления корзины, административных форм, страницы авторизации и сообщений об ошибках. Единое оформление делает приложение более понятным и удобным. JavaScript используется для добавления интерактивности. В рамках проекта он может применяться для обработки пользовательских действий на стороне клиента, улучшения поведения форм, работы с элементами интерфейса и выполнения небольших динамических операций. При этом основная бизнес-логика интернет-магазина остаётся на сервере, что делает систему более контролируемой. Особенностью проекта является то, что JavaScript используется не только в браузере, но и на сервере, так как серверная часть реализована на Node.js. Это удобно для разработки, поскольку один язык применяется как для клиентской, так и для серверной логики. Такой подход упрощает сопровождение проекта и делает структуру приложения более понятной. Таким образом, HTML, CSS и JavaScript формируют основу пользовательской части интернет-магазина цветов. HTML определяет структуру страниц, CSS отвечает за внешний вид, а JavaScript добавляет интерактивность и используется в серверной логике проекта.

### **1.3 Использование редактора кода**

#### **1.3.1 редактор кода Visual Studio Code**

Для разработки интернет-магазина цветов используется редактор кода Visual Studio Code. Это современная среда разработки, которая подходит для создания веб-приложений на JavaScript, Node.js, HTML, CSS и SQL. В рамках данного проекта VS Code удобен тем, что позволяет работать со всеми частями приложения в одном окне. Проект интернет-магазина состоит из нескольких каталогов и файлов. В нём присутствуют серверные файлы, маршруты, контроллеры, модели, шаблоны EJS, CSS-файлы, изображения, middleware и SQL-скрипты. Visual Studio Code позволяет удобно перемещаться между этими файлами, видеть структуру проекта и быстро находить нужные участки кода. Одним из преимуществ VS Code является

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | KP-09.02.07-B-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 11   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

подсветка синтаксиса. Она помогает отличать HTML-разметку, CSS-правила, JavaScript-код и SQL-запросы. Это особенно важно при работе с EJS-шаблонами, где HTML совмещается со вставками JavaScript. Подсветка делает код более читаемым и снижает вероятность ошибок. Также редактор поддерживает автодополнение. При написании JavaScript-кода это помогает быстрее работать с функциями, объектами, методами Express и подключаемыми модулями. Для проекта на Node.js такая возможность полезна при написании маршрутов, контроллеров и моделей. Встроенный терминал VS Code позволяет запускать сервер прямо из редактора. Разработчик может установить зависимости, запустить приложение, увидеть ошибки и сразу исправить их в коде. Это ускоряет процесс разработки и тестирования интернет-магазина. При работе над проектом VS Code также удобен для редактирования файлов базы данных. SQL-скрипты можно открыть рядом с моделями, чтобы сопоставлять названия таблиц и полей с запросами в коде. Это важно для корректной работы с пользователями, товарами, корзиной и заказами. Таким образом, Visual Studio Code является удобной средой разработки для интернет-магазина цветов. Он помогает организовать работу с проектом, редактировать разные типы файлов, запускать сервер и быстрее находить ошибки

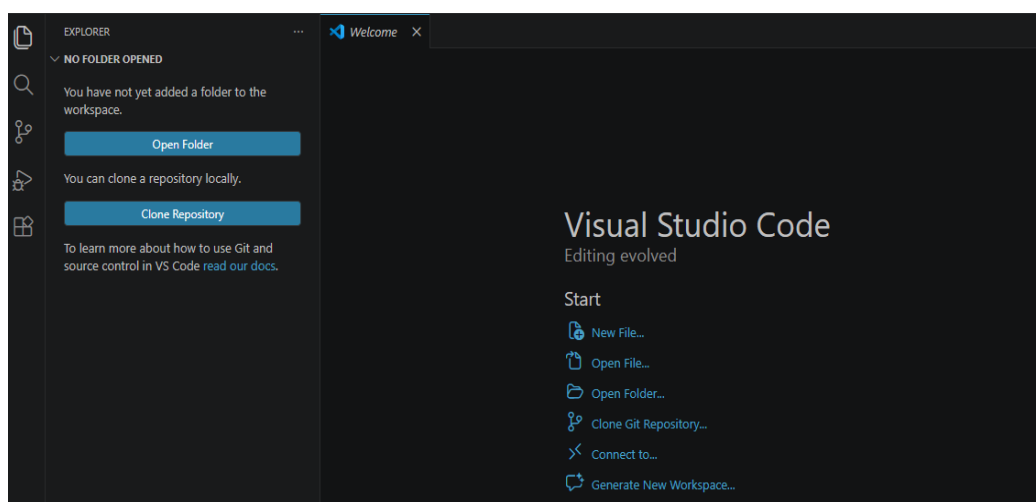


Рисунок 1 Редактор кода Visual Studio Code

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | KP-09.02.07-B-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 12   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

### 1.3.2 Клиент-серверная архитектура веб-приложения

Разрабатываемый интернет-магазин цветов построен на основе клиент-серверной архитектуры. Данная архитектура предполагает разделение приложения на две основные части: клиентскую и серверную. Клиентская часть — это браузер пользователя. Через него покупатель открывает сайт, просматривает каталог цветов, выбирает букет, добавляет товар в корзину, заполняет формы регистрации, входа и оформления заказа. Браузер отображает полученные от сервера HTML-страницы, стили и изображения. Серверная часть отвечает за обработку запросов. Когда пользователь открывает каталог, сервер получает запрос, обращается к базе данных, получает список товаров и формирует страницу. Когда пользователь добавляет букет в корзину, сервер проверяет данные и сохраняет выбранную позицию. Когда пользователь оформляет заказ, сервер создаёт соответствующие записи в базе данных. Обмен между клиентом и сервером происходит по HTTP. Для получения страниц используются GET-запросы. Например, пользователь может открыть главную страницу, каталог или карточку товара. Для отправки данных применяются POST-запросы. Они используются при регистрации, входе, добавлении товара в корзину, оформлении заказа и создании товара администратором. Клиент-серверная архитектура позволяет централизованно управлять важными данными. Пользователь не может напрямую изменить цену товара, список заказов или роль своей учётной записи. Все такие действия проходят через сервер, где выполняются проверки. Это особенно важно для интернет-магазина, так как система должна обеспечивать корректность данных и защищать административные функции. В проекте серверная часть реализована на Node.js с использованием Express. Сервер принимает запросы от браузера, передаёт их в нужные маршруты и контроллеры, взаимодействует с базой данных MySQL и возвращает пользователю результат. Страницы формируются с помощью EJS-шаблонов, поэтому пользователь получает

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | KP-09.02.07-B-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 13   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

готовый HTML-код. Таким образом, клиент-серверная архитектура обеспечивает разделение обязанностей. Браузер отвечает за отображение интерфейса, а сервер — за обработку логики, работу с базой данных, авторизацию, загрузку файлов и оформление заказов.

## **1.4 Выбор технологического стека**

### **1.4.1 Фреймворк Express в серверной части**

Express является основным фреймворком серверной части интернет-магазина цветов. Он используется вместе с Node.js и позволяет создавать маршруты, подключать middleware, обрабатывать HTTP-запросы и формировать ответы пользователю. В проекте Express выполняет роль центрального элемента серверной архитектуры. С его помощью настраивается приложение, подключаются статические файлы, шаблонизатор EJS, маршруты, обработка форм, загрузка файлов и обработчики ошибок. Благодаря Express отдельные части приложения объединяются в единую систему. Маршрутизация является одной из главных функций Express. Каждый маршрут отвечает за определённый раздел интернет-магазина. Например, отдельные маршруты могут использоваться для главной страницы, каталога цветов, карточки товара, регистрации и входа, корзины, оформления заказов и административной панели. Это позволяет разделить логику приложения по функциональным областям. Контроллеры обрабатывают действия, связанные с маршрутами. Например, контроллер каталога получает список товаров из базы данных и передаёт его в шаблон. Контроллер корзины добавляет выбранные букеты в корзину или удаляет их. Контроллер заказов оформляет покупку. Контроллер администратора отвечает за создание и редактирование товаров. Express также поддерживает middleware. Middleware — это промежуточные функции, которые выполняются между получением запроса и обработкой контроллером. В интернет-магазине они могут использоваться для проверки авторизации, проверки прав администратора, обработки ошибок, загрузки

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 14   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

файлов и обработки данных форм. Для интернет-магазина цветов middleware особенно важны при защите административной панели. Обычный пользователь не должен иметь доступ к управлению товарами и заказами. Поэтому перед переходом к административным маршрутам сервер должен проверить, авторизован ли пользователь и имеет ли он соответствующие права. Также Express используется для раздачи статических файлов. В проекте к статическим файлам относятся CSS, клиентский JavaScript, изображения и загруженные фотографии букетов. Благодаря этому браузер может корректно отображать оформление сайта и изображения товаров. Таким образом, Express обеспечивает работу всей серверной части интернет-магазина цветов. Он принимает запросы, распределяет их по маршрутам, подключает middleware, передаёт данные в шаблоны и связывает пользовательский интерфейс с базой данных.

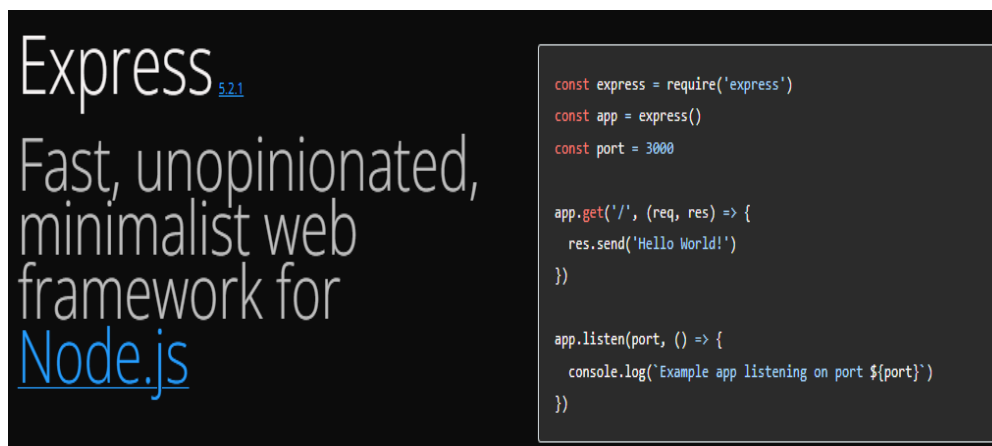


Рисунок 2 Главная страница Express js

### 1.4.2 EJS-шаблоны

Для формирования страниц интернет-магазина цветов используется шаблонизатор EJS. Он позволяет создавать HTML-страницы на сервере и вставлять в них данные, полученные из базы данных или контроллеров. EJS удобен для интернет-магазина, потому что большинство страниц зависят от динамических данных. Каталог должен отображать актуальный список цветов и букетов. Карточка товара должна показывать информацию о конкретном товаре. Корзина должна содержать товары, выбранные

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | KP-09.02.07-B-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 15   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

конкретным пользователем. Административная панель должна выводить список товаров и заказов. Всё это невозможно удобно реализовать только с помощью статического HTML. С помощью EJS сервер передаёт данные в шаблон, а шаблон формирует готовую страницу. Например, контроллер получает из MySQL список букетов и передаёт его в файл шаблона. Внутри EJS можно выполнить цикл и вывести каждую позицию как отдельную карточку товара. Если товаров нет, можно показать сообщение о пустом каталоге. EJS также позволяет использовать условия. Это полезно для отображения разных элементов интерфейса в зависимости от состояния пользователя. Например, если пользователь не авторизован, ему могут показываться ссылки на вход и регистрацию. Если пользователь вошёл в систему, ему доступны профиль, корзина и заказы. Если пользователь является администратором, ему дополнительно отображается ссылка на административную панель. Организация фронтенда в проекте строится на сочетании EJS, CSS, клиентского JavaScript и статических изображений. EJS отвечает за структуру и динамическое наполнение страниц. CSS задаёт внешний вид. JavaScript добавляет интерактивность. Изображения используются для визуального представления цветочной продукции. Для уменьшения повторения кода в EJS применяются частичные шаблоны. Например, шапка сайта и подвал могут быть вынесены в отдельные файлы и подключаться на разных страницах. Это удобно, потому что навигация используется почти во всех разделах сайта. Если нужно изменить меню, достаточно исправить один общий шаблон. В интернет-магазине цветов EJS применяется для отображения каталога, карточек товаров, корзины, форм входа и регистрации, страниц заказов и административных страниц. Такой подход позволяет связать данные из базы с пользовательским интерфейсом без создания отдельного клиентского приложения. Таким образом, EJS является важной частью фронтенда проекта. Он позволяет формировать динамические HTML-страницы на сервере, выводить данные о цветах и

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 16   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |



заказах, менять интерфейс в зависимости от роли пользователя и поддерживать единую структуру сайта.

### 1.4.3 Использование базы данных MySQL

Для хранения данных интернет-магазина цветов используется база данных MySQL. Это реляционная система управления базами данных, которая подходит для хранения структурированной информации. В проекте она применяется для хранения пользователей, товаров, корзины, заказов и другой информации, необходимой для работы магазина. Основная задача базы данных — обеспечить постоянное хранение данных. Если товар добавлен администратором, он должен сохраняться и отображаться в каталоге после перезапуска сервера. Если пользователь зарегистрировался, его данные должны быть доступны при следующем входе. Если покупатель оформил заказ, информация о заказе должна остаться в системе. Для интернет-магазина цветов в базе данных обычно выделяются несколько основных сущностей. Первая сущность — пользователи. В таблице пользователей хранятся имя, email, хеш пароля и роль. Роль нужна для разделения обычных покупателей и администратора. Вторая важная сущность — товары. В таблице товаров хранятся сведения о букетах и цветочных композициях: название, описание, цена, изображение и другие характеристики. Эти данные используются для вывода каталога и карточки товара. Третья сущность — корзина. Она хранит товары, которые пользователь выбрал перед оформлением заказа. Корзина связывает пользователя с выбранными товарами и количеством. Это позволяет каждому пользователю иметь собственный набор товаров. Четвёртая сущность — заказы. Когда пользователь оформляет покупку, содержимое корзины превращается в заказ. В заказе сохраняются сведения о пользователе, составе покупки, итоговой стоимости, адресе или другой информации, необходимой для обработки заявки. MySQL удобна тем, что позволяет связывать таблицы между собой. Например, заказ связан с пользователем, позиция заказа

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 17   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

связана с товаром, а корзина связана и с пользователем, и с товаром. Такие связи помогают сохранять целостность данных и делают структуру приложения логичной. В проекте работа с базой данных выполняется через серверную часть. Пользователь не обращается к MySQL напрямую. Он отправляет запрос через браузер, сервер обрабатывает его и выполняет нужный SQL-запрос. Это позволяет контролировать доступ к данным и предотвращать некорректные изменения. Для интернет-магазина база данных является одной из ключевых частей. Без неё сайт мог бы только показывать заранее написанную статическую информацию. Использование MySQL позволяет сделать магазин полноценным приложением, где данные можно добавлять, изменять, удалять и использовать в разных разделах сайта. Таким образом, MySQL обеспечивает хранение и обработку данных интернет-магазина цветов. Она используется для работы с пользователями, товарами, корзиной и заказами, а также обеспечивает связь между основными сущностями проекта.

#### 1.4.4 Хеширование паролей с использованием bcryptjs

Для работы интернет-магазина цветов требуется система регистрации и авторизации пользователей. Пользователь создаёт учётную запись, вводит email и пароль, после чего может входить в систему, добавлять товары в корзину и оформлять заказы. При этом пароль нельзя хранить в базе данных в открытом виде, так как это создаёт угрозу безопасности. Для защиты паролей используется библиотека bcryptjs. Она предназначена для хеширования паролей. Хеширование — это преобразование пароля в специальную строку, по которой нельзя напрямую восстановить исходное значение. В базу данных сохраняется не сам пароль, а его хеш. Когда пользователь регистрируется, сервер получает введённый пароль и перед сохранением обрабатывает его с помощью bcryptjs. После этого в таблицу пользователей записывается хеш. Даже если база данных окажется скомпрометирована, злоумышленник не увидит настоящие пароли

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 18   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

пользователей. При входе в систему процесс работает иначе. Пользователь вводит email и пароль. Сервер находит пользователя по email и сравнивает введенный пароль с сохранённым хешем. Для этого bcryptjs выполняет специальную проверку. Если пароль соответствует хешу, пользователь считается авторизованным. Если нет — вход запрещается. Такой подход является более безопасным, чем хранение паролей в обычном виде. Серверу не нужно знать исходный пароль пользователя после регистрации. Ему достаточно уметь проверить, совпадает ли введенное значение с сохранённым хешем. bcryptjs также применяется при смене пароля. Если пользователь меняет пароль в профиле, новый пароль снова хешируется, и только после этого обновляется запись в базе данных. Старый пароль при этом не сохраняется в открытом виде. Для интернет-магазина цветов безопасность авторизации важна, потому что с учётной записью связаны заказы, корзина и личные данные пользователя. Также через систему авторизации осуществляется разграничение доступа между покупателем и администратором. Администратор имеет больше прав, поэтому его пароль также должен быть защищён. Таким образом, bcryptjs обеспечивает безопасное хранение паролей пользователей. Его использование является важной частью серверной логики и позволяет защитить учётные записи в интернет-магазине цветов.

#### **1.4.5 Загрузка изображений товаров с использованием Multer**

В интернет-магазине цветов изображения товаров имеют большое значение. Покупатель выбирает букет или композицию в первую очередь по внешнему виду. Поэтому в проекте должна быть реализована возможность загружать фотографии товаров через административную часть. Для обработки загрузки файлов используется библиотека Multer. Она подключается к Express как middleware и предназначена для обработки форм, содержащих файлы. Обычная форма передаёт текстовые данные, например название товара, описание и цену. Если же форма содержит изображение,

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 19   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

запрос имеет тип multipart/form-data, и для его обработки требуется специальный инструмент. Multer принимает файл, отправленный через форму, сохраняет его на сервере и передаёт информацию о нём в контроллер. После этого сервер может записать путь к изображению в базу данных. Когда пользователь открывает каталог, этот путь используется в EJS-шаблоне для вывода изображения товара. В проекте загрузка файлов связана с административной панелью. Администратор добавляет новый букет, указывает его название, описание, цену и выбирает изображение. После отправки формы Multer сохраняет файл, а контроллер создаёт запись товара в базе данных. В дальнейшем загруженное изображение отображается в каталоге и карточке товара. Также Multer может использоваться при редактировании товара. Если администратор хочет заменить фотографию букета, он загружает новое изображение, и путь к нему обновляется в базе данных. Если изображение не меняется, товар может сохранить старую фотографию. При работе с загрузкой файлов важно учитывать безопасность. Сервер не должен принимать любые типы файлов. Для интернет-магазина цветов нужны только изображения, поэтому следует ограничивать допустимые форматы. Также полезно ограничивать размер файла, чтобы не перегружать сервер слишком большими изображениями. Multer позволяет настроить место хранения файлов и правила их именования. Это важно, потому что разные товары могут иметь изображения с одинаковыми исходными именами. Чтобы избежать конфликтов, файлам можно присваивать уникальные имена при загрузке. Таким образом, Multer обеспечивает работу с изображениями товаров. Для интернет-магазина цветов это обязательный функционал, так как без фотографий каталог не сможет полноценно представить букеты и композиции покупателю.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 20   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

#### 1.4.6 Связь используемых технологий с функционалом интернет-магазина

Все рассмотренные технологии применяются в проекте для выполнения конкретных задач. HTML, CSS и JavaScript отвечают за внешний вид и поведение пользовательского интерфейса. Express обеспечивает серверную часть, маршрутизацию и обработку запросов. EJS используется для формирования страниц на основе данных. MySQL хранит информацию о пользователях, товарах, корзине и заказах. Multer позволяет загружать изображения букетов. bcryptjs защищает пароли пользователей. Например, при просмотре каталога пользователь открывает страницу в браузере. Express принимает запрос, контроллер обращается к MySQL и получает список товаров. Затем данные передаются в EJS-шаблон, где формируются карточки букетов. CSS оформляет карточки, а браузер показывает пользователю готовый каталог. При добавлении товара администратором также взаимодействуют несколько технологий. HTML-форма позволяет ввести данные о букете. Multer обрабатывает загруженное изображение. Express передаёт запрос в контроллер. Контроллер сохраняет информацию о товаре в MySQL. После этого товар становится доступен в каталоге. При регистрации пользователя сервер получает данные из формы, проверяет их и хеширует пароль с помощью bcryptjs. Затем информация сохраняется в базе данных. При последующем входе сервер сравнивает введенный пароль с хешем и определяет, можно ли разрешить доступ. При оформлении заказа сервер получает содержимое корзины, связывает его с пользователем и сохраняет заказ в базе данных. После этого администратор может просмотреть поступивший заказ в административной панели. Такой сценарий показывает, что проект является не просто набором страниц, а полноценным веб-приложением с обработкой данных. Таким образом, каждая технология в проекте имеет практическое назначение. Они используются не абстрактно, а для реализации конкретных функций

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 21   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

интернет-магазина цветов: отображения каталога, управления товарами, загрузки изображений, регистрации пользователей, работы с корзиной и оформления заказов.

## 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ ИНТЕРНЕТ-МАГАЗИНА ЦВЕТОВ

### 2.1. Назначение и функциональные возможности разработанного приложения

Практическая часть курсовой работы посвящена разработке веб-приложения «Интернет-магазин цветов». Назначение системы заключается в автоматизации основных процессов онлайн-продажи цветочной продукции: отображении ассортимента, просмотре подробной информации о товаре, добавлении выбранных позиций в корзину, оформлении заказа, регистрации пользователей и администрировании содержимого магазина. Разработанное приложение ориентировано на два типа пользователей: обычного покупателя и администратора. Покупатель взаимодействует с публичной частью сайта. Он может открыть главную страницу, перейти в каталог товаров, просмотреть карточку конкретного букета или цветочной композиции, добавить товар в корзину, оформить заказ, зарегистрироваться, войти в личный кабинет и просмотреть свои заказы. Администратор, в свою очередь, получает доступ к служебной части сайта, где может управлять товарами, категориями, акциями и заказами. Функциональные возможности приложения включают:

- Просмотр главной страницы интернет-магазина.
- Просмотр каталога цветочной продукции.
- Просмотр подробной страницы товара.
- Регистрацию и авторизацию пользователей.
- Работу с личным кабинетом пользователя.
- Добавление товаров в корзину.
- Изменение количества товаров в корзине.
- Удаление товаров из корзины.
- Оформление заказа.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 23   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

- Просмотр заказов пользователя.
- Управление товарами через административную панель.
- Управление категориями товаров.
- Управление акциями и специальными предложениями.
- Просмотр заказов администратором.
- Загрузку изображений товаров.

В отличие от простого сайта-витрины, разработанное приложение имеет серверную логику и базу данных. Это означает, что товары, пользователи, корзина, заказы и акции не прописаны вручную в HTML-страницах, а хранятся в базе данных и выводятся динамически. Такой подход делает систему более гибкой, так как администратор может изменять ассортимент без изменения исходного кода страниц. Основной пользовательский сценарий начинается с просмотра каталога. Покупатель выбирает интересующий букет, открывает подробную страницу товара, оценивает изображение, цену и описание, после чего добавляет товар в корзину. Далее он переходит к оформлению заказа. Если пользователь не авторизован, система требует выполнить вход или регистрацию. После оформления заказа информация сохраняется в базе данных и становится доступной пользователю в профиле, а администратору — в административной панели. Таким образом, разработанное приложение представляет собой полноценный учебный интернет-магазин цветов, в котором реализованы основные функции электронной торговли.

## **2.2 Обоснование применения выбранного технологического стека**

### **2.2.1 Серверная платформа Node.js**

Серверная часть интернет-магазина реализована на платформе Node.js. Данная платформа позволяет выполнять JavaScript-код на стороне сервера. Это удобно при разработке веб-приложения, так как один язык программирования используется и для клиентской части, и для серверной логики. Node.js подходит для интернет-магазина цветов, потому что

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 24   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |



приложение должно обрабатывать большое количество разных запросов: открытие страниц, отправку форм, загрузку файлов, добавление товаров в корзину, оформление заказов и проверку пользователя. Асинхронная модель Node.js позволяет эффективно выполнять операции, связанные с обращением к базе данных и файловой системе. В проекте Node.js используется как основа серверного приложения. На нём запускается главный файл приложения, подключаются зависимости, создаётся сервер Express, настраиваются маршруты, шаблонизатор, статические файлы, сессии, обработчики ошибок и соединение с базой данных. Использование Node.js является обоснованным, так как проект написан на JavaScript, имеет серверную структуру и требует обработки HTTP-запросов.

## **2.3. Обоснование применения выбранного технологического стека**

### **2.3.1. Фреймворк Express**

Для построения серверной части используется фреймворк Express. Он является основным инструментом маршрутизации и обработки запросов. В проекте Express применяется для создания приложения, подключения middleware, обработки форм, раздачи статических файлов и передачи данных в EJS-шаблоны. Express позволяет разделить проект на отдельные маршруты. В приложении присутствуют маршруты для главной страницы, товаров, авторизации, корзины, заказов, профиля, акций и административной панели. Такое разделение делает структуру проекта понятной и удобной для сопровождения. Например, маршруты товаров отвечают за отображение каталога и подробной страницы товара. Маршруты корзины обрабатывают добавление товара, изменение количества и удаление позиций. Маршруты заказов используются для оформления покупки. Административные маршруты отвечают за управление товарами, категориями, акциями и заказами. Применение Express обосновано следующими причинами:

- простая настройка серверного приложения;
- удобная маршрутизация;

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 25   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

- поддержка middleware;
- возможность работы с EJS-шаблонами;
- удобная раздача статических файлов;
- совместимость с библиотеками для сессий, валидации, загрузки файлов работы с базой данных.

Таким образом, Express является центральной частью серверной архитектуры интернет-магазина цветов.

#### 2.3.4. Шаблонизатор EJS

Для формирования пользовательского интерфейса используется шаблонизатор EJS. Он позволяет создавать HTML-страницы на сервере и вставлять в них данные, полученные из базы данных. В проекте EJS применяется для отображения главной страницы, каталога товаров, подробной страницы товара, корзины, оформления заказа, личного кабинета, заказов пользователя, страниц авторизации и административной панели. Это позволяет формировать страницы динамически, без ручного создания отдельного HTML-файла для каждого товара. Например, каталог товаров строится на основе массива данных, полученного из базы. Шаблон перебирает товары и выводит их в виде карточек. Если администратор добавляет новый букет, он появляется в каталоге после сохранения в базе данных, а шаблон автоматически отображает его вместе с остальными товарами. EJS также используется для подключения общих частей интерфейса. В проекте есть частичные шаблоны, например head, header, footer, flash, product-card. Это позволяет избежать повторения одинакового HTML-кода на разных страницах. Шапка сайта, подключение стилей и карточка товара оформляются один раз и затем используются в нескольких представлениях.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 26   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

```

views > partials > <> header.ejs > ...
1  <header class="site-header">
2    <div class="container header-inner">
3      <a href="/" class="logo">Flower Shop</a>
4      <nav class="nav">
5        <a href="/">Главная</a>
6        <a href="/products">Каталог</a>
7        <a href="/promotions">Акции</a>
8        <a href="/cart">Корзина (<%= cartCount %>)</a>
9
10       <% if (currentUser) { %>
11         <a href="/profile">Профиль</a>
12         <% if (currentUser.role === 'admin') { %>
13           <a href="/admin">Админ</a>
14         <% } %>
15         <form action="/auth/logout" method="POST" class="inline-form">
16           <button type="submit" class="btn btn-link">Выйти</button>
17         </form>
18       <% } else { %>
19         <a href="/auth/login">Вход</a>
20       <% } %>
21     </nav>
22   </div>
23 </header>

```

Рисунок 3 Часть кода шаблонизатора EJS

### 2.3.5. База данных MySQL

Для хранения данных используется реляционная база данных MySQL. В интернет-магазине цветов база данных необходима для хранения пользователей, товаров, категорий, корзины, заказов, позиций заказов и акций. Использование MySQL обосновано тем, что данные интернет-магазина имеют чёткую структуру и связи. Пользователь может иметь несколько заказов, заказ может содержать несколько товаров, товар может относиться к категории, а корзина связывает пользователя с выбранными товарами. Такие отношения удобно представлять в виде таблиц. В проекте также присутствует SQL-файл инициализации базы данных. Он используется для создания структуры таблиц и подготовки базы к работе приложения. Подключение к базе данных вынесено в отдельный конфигурационный файл, что позволяет не смешивать настройки подключения с бизнес-логикой контроллеров. Работа с базой данных выполняется через модели. В проекте

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | KP-09.02.07-B-63-26ПЗ | Лист |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       | 27   |

присутствуют модели пользователя, товара, корзины, заказа и акций. Такой подход отделяет SQL-запросы от контроллеров. Контроллер отвечает за обработку запроса и выбор страницы, а модель — за получение, создание, изменение или удаление данных.

В приложении А предоставлена ER-Диаграмма базы данных.

#### **2.2.6. Дополнительные библиотеки**

Помимо основных технологий, в проекте используются дополнительные библиотеки, которые обеспечивают прикладные функции интернет-магазина. Для регистрации и входа пользователей применяется библиотека семейства bcrypt. Она используется для хеширования паролей. Благодаря этому пароли не сохраняются в базе данных в открытом виде. Для работы с сессиями используется механизм Express-сессий. Он позволяет сохранять состояние пользователя между запросами. Это необходимо для авторизации, работы корзины и вывода сообщений пользователю. Для хранения сессий используется MySQL-хранилище. Это позволяет сохранять данные сессий не только в памяти сервера, но и в базе данных. Для вывода временных сообщений используется connect-flash. Такие сообщения могут показываться после успешной регистрации, ошибки входа, добавления товара в корзину или выполнения административного действия. Для проверки данных форм используется express-validator. Он позволяет проверять корректность email, длину пароля, обязательность полей, числовые значения и другие параметры. Для загрузки изображений товаров используется Multer. Он принимает файлы из форм администратора и сохраняет их в каталог загрузок. Для имитации HTTP-методов PUT и DELETE из HTML-форм используется method-override. Это удобно для операций редактирования и удаления, так как обычные HTML-формы поддерживают только GET и POST. Также применяется dotenv, который позволяет хранить настройки приложения в переменных окружения.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 28   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

## 2.4. Архитектура и структура проекта

### 2.4.1. Общая архитектурная модель

Разработанное приложение имеет модульную архитектуру. Код разделён на несколько каталогов, каждый из которых отвечает за отдельную часть системы. Такая структура делает проект более понятным и позволяет изменять одну часть приложения, не затрагивая остальные. В проекте используются следующие основные каталоги:

- config — настройки подключения к базе данных, инициализация базы и хранилище сессий;
- controllers — контроллеры, содержащие основную бизнес-логику;
- models — модели для работы с базой данных;
- routes — маршруты приложения;
- middleware — промежуточные обработчики;
- views — EJS-шаблоны страниц;
- public — статические файлы;
- public/uploads — загруженные изображения товаров;
- sql — SQL-скрипт инициализации базы данных;
- utils — вспомогательные функции.

Такая организация соответствует классическому подходу MVC, где маршруты принимают запросы, контроллеры обрабатывают действия пользователя, модели работают с данными, а представления формируют интерфейс.

### 2.4.2. Маршруты приложения

Маршруты расположены в каталоге routes. Каждый файл маршрутов отвечает за отдельный раздел интернет-магазина. Файл index.js отвечает за главную страницу сайта. Через него пользователь попадает на стартовую страницу интернет-магазина, где может увидеть основные элементы витрины

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 29   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

и перейти к каталогу. Файл `products.js` отвечает за товары. Он используется для вывода каталога и подробной страницы конкретного товара. Эти маршруты являются одними из основных, так как каталог — центральный раздел пользовательской части магазина. Файл `auth.js` отвечает за регистрацию, вход и выход пользователя. Через эти маршруты пользователь создаёт учётную запись, проходит авторизацию и завершает сеанс. Файл `cart.js` реализует работу с корзиной. Он обрабатывает просмотр корзины, добавление товара, изменение количества и удаление позиции. Файл `orders.js` связан с оформлением заказов. Он используется для перехода к оформлению, сохранения заказа и вывода страницы успешного оформления. Файл `profile.js` отвечает за личный кабинет пользователя. В нём реализованы маршруты просмотра профиля, редактирования данных и просмотра заказов пользователя. Файл `promotions.js` отвечает за страницу акций. Это позволяет вынести специальные предложения магазина в отдельный раздел. Файл `admin.js` отвечает за административную панель. Он используется для управления товарами, категориями, заказами и акциями. Доступ к нему должен быть ограничен только для администратора.

```

1  const express = require('express');
2  const ProductController = require('../controllers/productController');
3  const productRoutes = require('./products');
4  const authRoutes = require('./auth');
5  const cartRoutes = require('./cart');
6  const profileRoutes = require('./profile');
7  const orderRoutes = require('./orders');
8  const promotionRoutes = require('./promotions');
9  const adminRoutes = require('./admin');
10
11  const router = express.Router();
12
13  router.get('/', ProductController.renderHomePage);
14  router.use('/products', productRoutes);
15  router.use('/auth', authRoutes);
16  router.use('/cart', cartRoutes);
17  router.use('/profile', profileRoutes);
18  router.use('/orders', orderRoutes);
19  router.use('/promotions', promotionRoutes);
20  router.use('/admin', adminRoutes);
21
22  module.exports = router;
23

```

Рисунок 4 Часть кода маршрутизатора

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 30   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

### 2.4.3. Контроллеры как центр бизнес-логики

Контроллеры расположены в каталоге `controllers`. Они содержат основную прикладную логику интернет-магазина. `productController.js` отвечает за работу с товарами в пользовательской части. Он получает список товаров, передаёт данные в шаблон каталога и формирует страницу подробного просмотра товара. `authController.js` отвечает за регистрацию, вход и выход пользователя. В нём реализованы проверка введённых данных, поиск пользователя в базе, хеширование пароля и создание пользовательской сессии. `cartController.js` реализует работу корзины. Он добавляет товар в корзину, выводит содержимое корзины, изменяет количество товара и удаляет позиции. `orderController.js` отвечает за оформление заказа. Он получает данные из корзины, создаёт заказ, сохраняет его позиции и очищает корзину после успешного оформления. `profileController.js` отвечает за личный кабинет. Он выводит информацию о пользователе, позволяет изменять личные данные и просматривать историю заказов. `promotionController.js` отвечает за вывод акций. Через него пользователь может просматривать специальные предложения магазина. `adminController.js` содержит административную логику. Он используется для управления товарами, категориями, заказами и акциями. Разделение контроллеров по функциям делает код более понятным. Например, логика корзины не смешивается с логикой авторизации, а управление товарами в административной панели отделено от пользовательского каталога.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 31   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

```

class CartController {
  static async renderCartPage(req, res, next) {
    try {
      const data = req.session.user
        ? await CartModel.getDbCartByUserId(req.session.user.id)
        : await CartModel.getSessionCartDetailed(req.session.cart || []);

      res.render('cart/index', {
        title: 'Корзина',
        cartItems: data.items,
        subtotal: data.subtotal,
        discount: 0,
        total: data.subtotal
      });
    } catch (error) {
      next(error);
    }
  }
}

```

Рисунок 5 Код контроллера корзины

#### 2.4.4. Middleware в проекте

Middleware расположены в каталоге middleware. Они выполняют промежуточную обработку запросов и позволяют вынести повторяющуюся логику в отдельные файлы. authMiddleware.js используется для проверки авторизации пользователя. Он защищает маршруты, к которым должен иметь доступ только вошедший пользователь, например корзину, оформление заказа и профиль. adminMiddleware.js проверяет права администратора. Он используется для ограничения доступа к административной панели. Обычный пользователь не должен иметь возможности управлять товарами или заказами. uploadMiddleware.js отвечает за обработку загрузки изображений товаров. Он использует Multer и применяется в административной части при создании или редактировании товара. validation.js содержит правила проверки данных форм. Валидация нужна для регистрации, входа, создания товаров, оформления заказа и других действий. errorHandler.js реализует централизованную обработку ошибок. Если в приложении возникает ошибка, она передаётся в общий обработчик, который формирует ответ пользователю. notFound.js отвечает за обработку

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 32   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |



несуществующих страниц. Если пользователь переходит по адресу, для которого нет маршрута, приложение отображает страницу ошибки. Использование middleware делает код контроллеров более компактным и снижает дублирование.

```
module.exports = (err, req, res, next) => {
  const normalizedError = mapMySqlError(err);
  const statusCode = normalizedError.statusCode || 500;
  const safeMessage = statusCode >= 500
    ? 'На сервере произошла ошибка. Попробуйте ещё раз позже.'
    : normalizedError.message || 'Не удалось выполнить операцию.';

  console.error(normalizedError);

  if (req.accepts('html')) {
    return res.status(statusCode).render('error', {
      title: 'Ошибка',
      statusCode,
      message: safeMessage
    });
  }

  return res.status(statusCode).json({
    message: safeMessage,
    details: statusCode < 500 ? normalizedError.details || null : null
  });
};
```

Рисунок 6 Часть кода middleware

## 2.5. Модель данных и организация базы данных

### 2.5.1 Общая структура базы данных

База данных интернет-магазина цветов описана в SQL-скрипте, расположенном в каталоге sql. Она предназначена для хранения всех основных данных приложения: пользователей, товаров, категорий, корзины, заказов, позиций заказов, акций и сессий. Основными сущностями системы являются:

- пользователи;
- товары;
- категории;
- корзина;
- заказы;
- позиции заказов;

- акции;
- сессии.

Такая структура соответствует функционалу интернет-магазина. Пользователь может регистрироваться, добавлять товары в корзину и оформлять заказы. Товары могут относиться к категориям и участвовать в акциях. Заказы должны сохраняться отдельно от корзины, так как после оформления они становятся частью истории покупок.

## **2.6. Реализация регистрации, аутентификации и авторизации**

### **2.6.1. Регистрация пользователя**

Регистрация необходима для создания учётной записи покупателя. Пользователь заполняет форму регистрации, указывает имя, email и пароль. После отправки формы данные поступают на сервер. Сначала сервер выполняет валидацию. Проверяется корректность email, заполненность обязательных полей и соответствие пароля требованиям. Если данные заполнены неверно, пользователь получает сообщение об ошибке. Затем сервер проверяет, существует ли пользователь с таким email. Это необходимо, чтобы не допустить регистрацию двух аккаунтов с одинаковым адресом электронной почты. Если email свободен, пароль хешируется и новая запись сохраняется в базе данных. После успешной регистрации пользователь может войти в систему и использовать функции, доступные авторизованным пользователям.

### **2.6.2. Хеширование пароля**

Пароль пользователя не сохраняется в базе данных в открытом виде. Для его защиты применяется хеширование с использованием библиотеки bcrypt. При регистрации введенный пароль преобразуется в хеш. В базу данных записывается только результат преобразования. Это означает, что даже при получении доступа к базе данных нельзя увидеть исходные пароли пользователей. При входе в систему сервер сравнивает введенный пароль с сохранённым хешем. Если сравнение успешно, пользователь считается

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 34   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

подлинным. Если пароль не совпадает, авторизация отклоняется. Такой механизм является обязательным для интернет-магазина, поскольку учётная запись пользователя связана с корзиной, заказами и личными данными.

```
const passwordHash = await bcrypt.hash(password, 10);
const userId = await UserModel.createUser({ fullName, email, passwordHash, phone, address });
req.session.user = await UserModel.getUserById(userId);

await syncSessionCartToDb(userId, req.session.cart || []);
req.session.cart = [];

req.flash('success', 'Регистрация выполнена успешно.');
res.redirect('/profile');
} catch (error) {
  next(error);
}
```

Рисунок 7 Код хэширования пароля

### 2.6.3. Вход пользователя

Вход в систему выполняется через форму авторизации. Пользователь вводит email и пароль. Сервер ищет пользователя по email и проверяет пароль. Если пользователь найден и пароль указан верно, сервер создаёт пользовательскую сессию. В сессии сохраняется информация, позволяющая определить текущего пользователя при последующих запросах. После успешного входа пользователь может работать с корзиной, оформлять заказы и просматривать личный кабинет. Если пользователь является администратором, ему становится доступна административная панель. Если данные введены неправильно, приложение выводит сообщение об ошибке и не предоставляет доступ к защищённым разделам.

В приложении Б диаграмма последовательности сценария входа пользователя.

### 2.6.4. Выход из системы

Выход из системы завершает пользовательскую сессию. После выхода пользователь снова становится гостем. Ему остаются доступны публичные страницы, например главная страница, каталог и карточки товаров, но

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 35   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

закрытые разделы становятся недоступны. Этот механизм нужен для безопасности, особенно если пользователь работает за общим компьютером.

### **2.6.5. Разграничение прав доступа**

В проекте реализовано разграничение прав доступа. Пользователи делятся на две основные категории:

- обычный пользователь;
- администратор.

Обычный пользователь может просматривать каталог, добавлять товары в корзину, оформлять заказы и работать с профилем. Администратор дополнительно может управлять товарами, категориями, акциями и заказами. Проверка доступа выполняется через middleware. Если пользователь не авторизован, он не может перейти к корзине, оформлению заказа или профилю. Если пользователь не является администратором, он не может открыть административную панель. Такое разграничение является важным элементом безопасности интернет-магазина.

## **2.7. Реализация каталога товаров**

### **2.7.1. Назначение каталога**

Каталог является центральной частью пользовательского интерфейса интернет-магазина цветов. Через него покупатель знакомится с ассортиментом, выбирает интересующие букеты и переходит к подробной информации. Каталог строится динамически. Сервер получает список товаров из базы данных и передаёт его в EJS-шаблон. Шаблон выводит товары в виде карточек. Каждая карточка содержит изображение, название, цену и ссылку на подробную страницу. Для цветочного магазина каталог особенно важен, потому что покупатель принимает решение в первую очередь на основе внешнего вида товара. Поэтому карточки должны быть визуально понятными и аккуратными.

В приложении В предоставлена диаграмма последовательности для просмотра каталога товаров пользователем.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 36   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

### 2.7.2. Карточка товара в каталоге

Карточка товара используется для краткого представления букета или композиции. Она содержит основные сведения, необходимые пользователю для первичного выбора. В карточке отображаются:

- изображение товара;
- название;
- цена;
- кнопка или ссылка для перехода к подробной странице.

В проекте карточка товара вынесена в отдельный частичный шаблон `product-card.ejs`. Это позволяет использовать один и тот же внешний вид карточки на разных страницах. Если нужно изменить оформление карточки, достаточно изменить один файл. Такой подход уменьшает дублирование кода и делает интерфейс более единообразным.

### 2.7.3. Подробная страница товара

Подробная страница товара открывается при выборе конкретной позиции из каталога. На ней пользователь получает больше информации о букете или цветочной композиции. На странице товара отображаются:

- название товара;
- изображение;
- описание;
- цена;
- форма добавления в корзину.

Подробная страница нужна для того, чтобы пользователь мог принять окончательное решение перед покупкой. В цветочном магазине описание товара может содержать сведения о составе букета, назначении, оформлении или особенностях композиции. После изучения информации пользователь может добавить товар в корзину. Если для этого требуется авторизация,

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       |      |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       | 37   |

система должна перенаправить пользователя на вход или показать соответствующее сообщение.

## **2.8. Реализация каталога товаров**

### **2.8.1 Добавление товара в корзину**

Корзина предназначена для временного хранения товаров, которые пользователь планирует заказать. Она является промежуточным этапом между просмотром каталога и оформлением заказа. В интернет-магазине цветов корзина позволяет собрать несколько позиций в один заказ. Например, пользователь может выбрать один основной букет и дополнительно добавить ещё одну композицию. Все выбранные товары отображаются в корзине с указанием количества и общей стоимости. Корзина связана с пользователем, поэтому для полноценной работы с ней требуется авторизация.

В приложении Д рисунок корзины

### **2.8.2. Добавление товара в корзину**

Добавление товара начинается с подробной страницы товара. Пользователь нажимает кнопку добавления, после чего браузер отправляет запрос на сервер. Сервер получает идентификатор товара и проверяет, существует ли такой товар в базе данных. Если товар найден, он добавляется в корзину пользователя. Если такая позиция уже есть в корзине, количество может быть увеличено. Такой подход удобен для покупателя, потому что повторное добавление одного и того же товара не создаёт лишние строки, а изменяет количество.

### **2.8.3. Просмотр корзины**

При открытии корзины сервер получает все позиции, связанные с текущим пользователем. Затем данные передаются в EJS-шаблон, который формирует страницу корзины. На странице корзины отображаются:

- список выбранных товаров;
- изображение и название товара;

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 38   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

- цена;
- количество;
- сумма по каждой позиции;
- итоговая сумма заказа;
- кнопки изменения или удаления;
- переход к оформлению заказа.

Корзина позволяет пользователю проверить заказ перед оформлением. Это снижает вероятность ошибки, когда покупатель случайно выбирает не тот товар или неверное количество.

#### **2.8.4. Изменение количества и удаление товара**

Пользователь может изменить количество товара в корзине. Для этого отправляется запрос на сервер, после чего соответствующая запись обновляется в базе данных. Также пользователь может удалить товар из корзины. В этом случае запись удаляется из таблицы корзины, а итоговая сумма пересчитывается. Такая функциональность делает корзину удобной и соответствует обычному поведению интернет-магазинов.

### **2.9. Реализация оформления заказа**

#### **2.9.1. Назначение подсистемы заказов**

Оформление заказа является завершающим этапом пользовательского сценария покупки. До этого момента товары находятся только в корзине. После оформления они становятся постоянной записью в базе данных. Подсистема заказов нужна для фиксации покупки, сохранения состава заказа и передачи информации администратору. Без этой подсистемы приложение было бы только каталогом с корзиной, но не полноценным интернет-магазином.

#### **2.9.2. Переход к оформлению**

Пользователь переходит к оформлению из корзины. На этом этапе сервер проверяет, есть ли товары в корзине. Если корзина пуста, оформление невозможно. Если товары есть, приложение открывает страницу оформления

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 39   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

заказа. На ней пользователь должен указать необходимые данные для обработки заказа. В зависимости от реализации это может быть адрес, телефон, комментарий или другая контактная информация.

### **2.9.3. Создание заказа**

После отправки формы оформления сервер получает данные пользователя и содержимое корзины. Затем создаётся запись заказа в базе данных. После этого для каждого товара из корзины создаётся отдельная позиция заказа. Такой подход позволяет сохранить структуру покупки. Один заказ может включать несколько товаров, поэтому состав заказа хранится отдельно от общей информации о заказе. После успешного создания заказа корзина очищается. Пользователь перенаправляется на страницу успешного оформления.

### **2.9.4. Страница успешного оформления**

После оформления заказа пользователь видит страницу подтверждения. Она сообщает, что заказ был создан успешно. Такая страница важна с точки зрения пользовательского опыта. Покупатель должен понимать, что его действие завершилось корректно и заказ был принят системой.

### **2.9.5. История заказов пользователя**

В личном кабинете пользователь может просмотреть свои заказы. Для этого сервер получает из базы данных заказы, связанные с текущим пользователем, и передаёт их в шаблон. История заказов позволяет пользователю видеть ранее оформленные покупки. Это особенно удобно, если покупатель регулярно заказывает цветы к праздникам или повторяет похожие заказы.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 40   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |



## **2.10. Реализация административной панели**

### **2.10.1. Назначение административной панели**

Административная панель предназначена для управления интернет-магазином. Она доступна только пользователю с ролью администратора. Через административную часть можно управлять товарами, категориями, заказами и акциями. Это позволяет изменять содержимое сайта без ручного редактирования кода и базы данных. Наличие административной панели делает проект более полноценным, так как интернет-магазин должен иметь не только пользовательскую витрину, но и инструменты управления.

### **2.10.2. Главная страница администратора**

Главная страница административной панели выполняет роль панели управления. С неё администратор может перейти к основным разделам: товарам, категориям, заказам и акциям. Такое разделение упрощает работу с магазином. Администратор видит не все функции сразу, а переходит в нужный раздел в зависимости от задачи.

### **2.10.3. Управление товарами**

Раздел управления товарами позволяет администратору просматривать список товаров, добавлять новые позиции, редактировать существующие и удалять ненужные. При добавлении товара администратор указывает название, описание, цену, категорию и загружает изображение. После сохранения данные попадают в базу данных, а изображение сохраняется в каталог загрузок. Редактирование товара позволяет изменить информацию о букете или композиции. Например, администратор может обновить цену, описание, категорию или фотографию. Удаление товара используется, если позиция больше не продаётся или была добавлена ошибочно.

В приложении Г предоставлена диаграмма последовательности добавления нового товара администратором.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       |      |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       | 41   |

#### **2.10.4. Управление категориями**

Категории помогают структурировать каталог. В административной панели предусмотрен отдельный раздел для работы с категориями. Администратор может добавлять новые категории, изменять существующие и удалять ненужные. Это удобно для цветочного магазина, потому что ассортимент может меняться в зависимости от сезона, праздников и состава товаров. Например, могут появляться категории для праздничных букетов, композиций, монобукетов или акционных предложений.

#### **2.10.5. Управление заказами**

Раздел заказов позволяет администратору просматривать оформленные пользователями заказы. Для каждого заказа отображается информация о покупателе, составе заказа, сумме, статусе и дате оформления. Эта функция необходима для обработки заявок. Администратор может видеть, какие товары заказаны, и организовывать дальнейшую работу магазина. В интернет-магазине цветов заказы особенно важны, так как цветочная продукция часто приобретается к конкретной дате или событию. Поэтому администратор должен иметь возможность быстро просматривать поступившие заявки.

#### **2.10.6. Управление акциями**

В проекте реализован отдельный раздел акций. Администратор может управлять специальными предложениями, которые отображаются пользователю на соответствующей странице. Акции могут использоваться для выделения сезонных предложений, праздничных букетов или скидок. Для цветочного магазина это особенно актуально, так как спрос на продукцию часто зависит от календарных дат. Наличие отдельного раздела акций расширяет функциональность проекта и делает его ближе к реальному интернет-магазину.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 42   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

## **2.11. Реализация профиля пользователя**

### **2.11.1. Назначение личного кабинета**

Личный кабинет предназначен для работы пользователя с собственными данными и заказами. Он становится доступен после авторизации. В профиле пользователь может просматривать информацию о своей учётной записи, редактировать личные данные и переходить к истории заказов. Наличие профиля делает учётную запись более полезной. Пользователь не просто входит в систему, а получает персональный раздел, связанный с его действиями в магазине.

### **2.11.2. Редактирование профиля**

В проекте предусмотрена страница редактирования профиля. Через неё пользователь может изменить личные данные. Сервер получает данные из формы, проверяет их корректность и обновляет запись пользователя в базе данных. После успешного изменения пользователь возвращается в профиль или получает сообщение об успешном обновлении. Редактирование профиля удобно для интернет-магазина, так как пользовательские данные могут использоваться при оформлении заказов.

### **2.11.3. Просмотр заказов в профиле**

Отдельный раздел профиля предназначен для просмотра заказов пользователя. Он получает данные из базы и выводит список оформленных заказов. Пользователь может видеть, какие товары он заказывал ранее. Это повышает удобство работы с приложением и делает систему более завершённой.

## **2.12. Реализация пользовательского интерфейса**

### **2.12.1. Общий подход к интерфейсу**

Пользовательский интерфейс реализован с использованием EJS, HTML, CSS и клиентского JavaScript. Страницы формируются на сервере, а затем отображаются в браузере пользователя. Интерфейс построен по шаблонному принципу. Общие элементы вынесены в частичные шаблоны:

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | KP-09.02.07-B-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 43   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

подключение заголовка страницы, шапка, подвал, flash-сообщения и карточка товара. Это уменьшает повторение кода и делает внешний вид сайта единым.

### **2.12.2. Шапка сайта и навигация**

Шапка сайта содержит основные элементы навигации. Через неё пользователь может перейти на главную страницу, в каталог, акции, корзину, профиль или войти в систему. Состав навигации зависит от состояния пользователя. Гость видит ссылки на вход и регистрацию. Авторизованный пользователь получает доступ к профилю, корзине и заказам. Администратор дополнительно видит ссылку на административную панель. Такой подход делает интерфейс адаптированным к роли пользователя.

### **2.12.3. Главная страница**

Главная страница выполняет роль стартовой витрины интернет-магазина. Она знакомит пользователя с сайтом и направляет его к основным разделам. На главной странице могут отображаться приветственный блок, ссылки на каталог, акции или популярные товары. Главная страница важна для первого впечатления пользователя.

### **2.12.4. Каталог и карточки товаров**

Каталог оформлен в виде набора карточек. Такое представление удобно для цветочного магазина, потому что позволяет показать сразу несколько букетов с изображениями и ценами. Карточка товара содержит только основные сведения. Если пользователь заинтересовался товаром, он переходит на подробную страницу. Единое оформление карточек делает каталог визуально аккуратным и удобным для просмотра.

### **2.12.5. Формы**

В проекте используются формы регистрации, входа, редактирования профиля, оформления заказа и административного управления товарами. Формы оформлены единообразно. Это важно для удобства пользователя: одинаковая структура полей и кнопок помогает быстрее понять, как работать

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 44   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

с сайтом. Для форм также предусмотрена обработка ошибок. Если пользователь вводит некорректные данные, приложение выводит сообщение.

### 2.12.6. Flash-сообщения

В проекте используются flash-сообщения. Они позволяют выводить пользователю краткие уведомления после выполнения действия. Например, сообщение может появиться после успешного входа, ошибки авторизации, добавления товара в корзину, оформления заказа или выполнения административной операции. Flash-сообщения делают интерфейс более понятным, потому что пользователь получает обратную связь от системы.

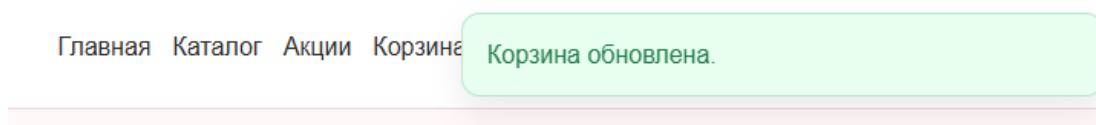


Рисунок 8 Flash-сообщение

## 2.13. Обработка ошибок и валидация данных

### 2.13.1. Валидация входных данных

Валидация данных выполняется с помощью middleware. Она необходима для проверки информации, которую пользователь отправляет через формы. В проекте проверяются данные регистрации, входа, оформления заказа, редактирования профиля и административных форм. Например, email должен иметь корректный формат, пароль должен соответствовать минимальным требованиям, название товара не должно быть пустым, цена должна быть числом. Валидация защищает приложение от некорректных данных и снижает вероятность ошибок при работе с базой данных.

### 2.13.2. Обработка ошибок

В проекте реализована централизованная обработка ошибок. Для этого используется специальный middleware errorHandler.js. Если в процессе работы возникает ошибка, она передаётся в общий обработчик. После этого приложение формирует ответ пользователю. Такой подход удобнее, чем обработка ошибок отдельно в каждом маршруте. Также в проекте есть

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 45   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

обработчик несуществующих страниц. Если пользователь переходит по неправильному адресу, приложение показывает страницу ошибки.

```
function mapMySqlError(err) {
  if (err.code === 'ER_DUP_ENTRY') {
    const match = err.sqlMessage?.match(/for key '([^']+)'/i);
    const key = match?.[1] || '';

    for (const [table, labels] of Object.entries(DUPLICATE_FIELD_LABELS)) {
      for (const [field, message] of Object.entries(labels)) {
        if (key.includes(table) && key.includes(field)) {
          return new HttpError(message, 400);
        }
      }
    }

    return new HttpError('Запись с такими данными уже существует.', 400);
  }

  if (err.code === 'ER_ROW_IS_REFERENCED_2' || err.code === 'ER_NO_REFERENCED_ROW_2') {
    return new HttpError('Операция не может быть выполнена из-за связанных данных в базе.', 400);
  }

  return err;
}
```

Рисунок 9 Код обработчика ошибок

## 2.14. Результаты работы программы

### 2.14.1. Практический результат реализации

В результате разработки было создано серверное веб-приложение интернет-магазина цветов. Оно включает пользовательскую часть, административную панель, базу данных, систему авторизации, корзину, заказы, загрузку изображений и раздел акций. Пользователь может:

- открыть главную страницу;
- просмотреть каталог цветов;
- открыть подробную страницу товара;
- зарегистрироваться;
- войти в систему;
- добавить товар в корзину;
- изменить содержимое корзины;
- оформить заказ;
- просмотреть свой профиль;
- просмотреть историю заказов;
- открыть страницу акций.

Администратор может:

- войти в административную панель;
- просматривать список товаров;
- добавлять товары;
- редактировать товары;
- удалять товары;
- управлять категориями;
- просматривать заказы;
- управлять акциями.

Таким образом, приложение реализует основные функции интернет-магазина и может использоваться как учебный пример веб-системы электронной торговли.

#### **2.14.2. Значение реализованных механизмов**

Наиболее важными результатами проекта являются:

- модульная структура приложения;
- использование Express для серверной части;
- применение EJS для динамического формирования страниц;
- хранение данных в MySQL;
- регистрация и авторизация пользователей;
- хеширование паролей;
- разграничение прав пользователя и администратора;
- работа с корзиной;
- оформление заказов;
- загрузка изображений товаров;
- административное управление ассортиментом;
- вывод flash-сообщений;
- обработка ошибок и валидация данных.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 47   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

Эти механизмы показывают, что проект является не просто статическим сайтом, а полноценным серверным веб-приложением.



## ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана информационная система в виде веб-приложения «Интернет-магазин цветочного салона». В процессе работы были изучены особенности предметной области, связанные с организацией онлайн-продажи цветочной продукции, а также определены основные требования к функциональности современного интернет-магазина.

В теоретической части были рассмотрены базовые технологии веб-разработки: HTML, CSS и JavaScript, изучены принципы клиент-серверной архитектуры, а также проанализированы возможности редактора Visual Studio Code. Особое внимание было уделено выбору технологического стека. Для реализации серверной части была использована платформа Node.js и фреймворк Express, для формирования пользовательского интерфейса — шаблонизатор EJS, для хранения данных — система управления базами данных MySQL. Дополнительно были рассмотрены библиотеки bcryptjs для защиты пользовательских паролей и Multer для загрузки изображений товаров.

В практической части разработано полнофункциональное веб-приложение, включающее пользовательскую и административную части. Реализованы основные возможности интернет-магазина: просмотр каталога товаров, отображение подробной информации о товаре, регистрация и авторизация пользователей, работа с корзиной, оформление заказов, просмотр истории заказов, а также управление товарами, категориями, заказами и акциями через административную панель.

В ходе разработки была спроектирована структура базы данных, обеспечивающая хранение информации о пользователях, товарах, категориях, корзине, заказах и акциях. Реализованы механизмы разграничения прав доступа, валидации данных, централизованной

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 49   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

обработки ошибок и безопасного хранения паролей. Также обеспечена возможность загрузки и хранения изображений цветочной продукции.

Поставленная цель — разработка веб-приложения «Интернет-магазин цветов» — была полностью достигнута. Все задачи, сформулированные во введении, успешно решены. Созданное приложение демонстрирует практическое применение современных технологий веб-разработки и может использоваться как учебный проект, а также как основа для дальнейшего развития и внедрения в реальных условиях коммерческой деятельности.

Практическая значимость работы заключается в создании готового программного продукта, который автоматизирует основные процессы интернет-торговли цветочной продукцией. Разработанная система может быть расширена дополнительными возможностями, такими как интеграция с платёжными сервисами, подключение службы доставки, внедрение системы уведомлений и аналитики пользовательского поведения.

Таким образом, результаты выполненной курсовой работы подтверждают эффективность выбранных технологий и архитектурных решений для создания современного веб-приложения электронной коммерции.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 50   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

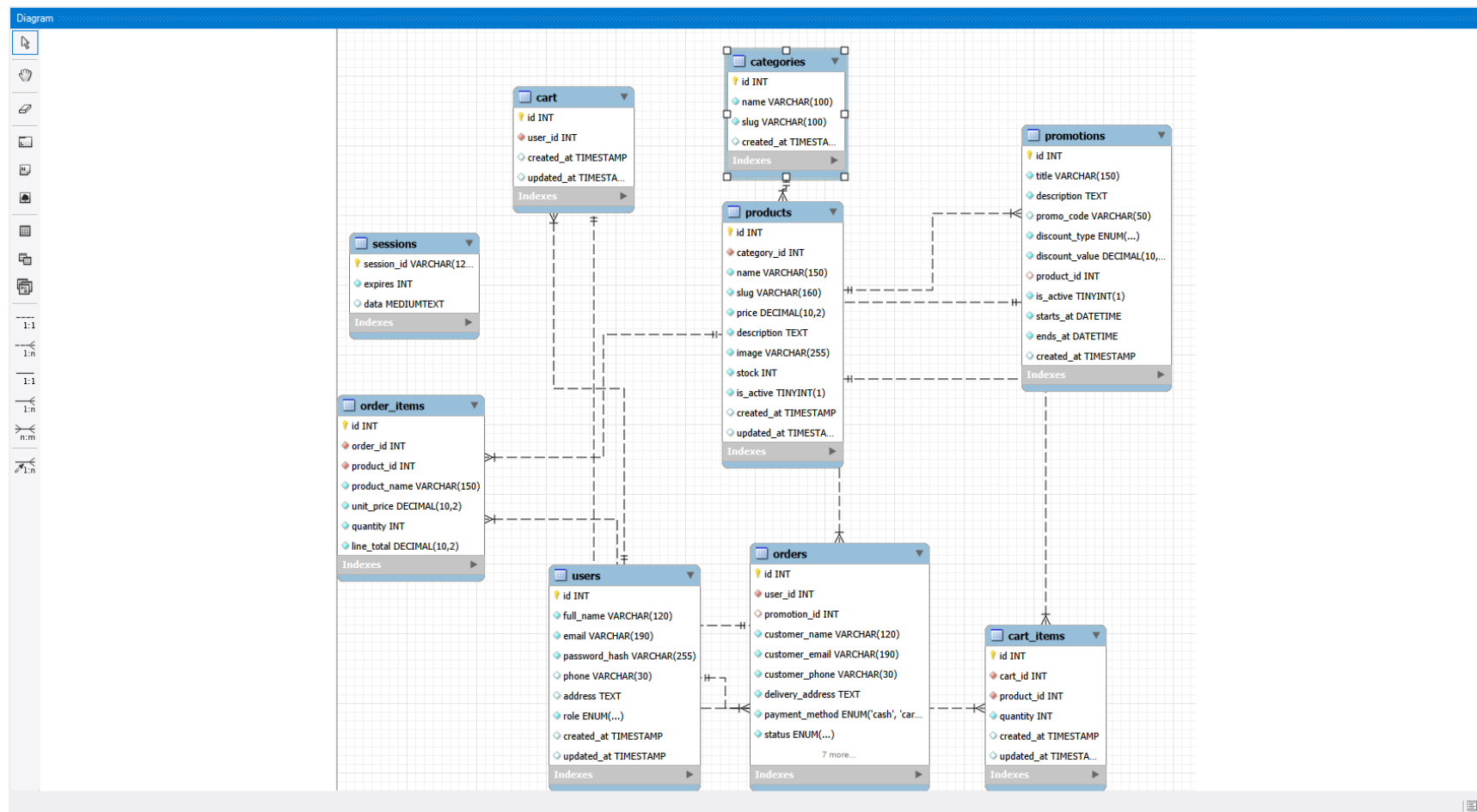
1. Дакетт, Д. HTML и CSS. Разработка и дизайн веб-сайтов / Д. Дакетт. — Москва : Эксмо, 2023. — 480 с.
2. Дакетт, Д. JavaScript и jQuery. Интерактивная веб-разработка / Д. Дакетт. — Москва : Эксмо, 2022. — 640 с.
3. Шилдт, Г. JavaScript. Полное руководство / Г. Шилдт. — Москва : Вильямс, 2021. — 800 с.
4. Тейт, Б. Node.js для профессионалов / Б. Тейт. — Санкт-Петербург : Питер, 2022. — 416 с.
5. Тилков, Ш. Node.js в действии / Ш. Тилков, С. Виноски. — Москва : ДМК Пресс, 2021. — 512 с.
6. Хоган, Б. Создание веб-приложений с использованием Node.js / Б. Хоган. — Санкт-Петербург : Питер, 2023. — 368 с.
7. Элмасри, Р. Системы баз данных. Полный курс / Р. Элмасри, Ш. Нават. — 7-е изд. — Москва : Вильямс, 2022. — 1360 с.
8. Карпова, Т. С. Базы данных: модели, разработка, реализация / Т. С. Карпова. — Москва : КноРус, 2023. — 496 с.
9. MDN Web Docs. Руководство по веб-разработке. — URL: <https://developer.mozilla.org> (дата обращения: 03.05.2026).
10. Node.js Documentation. — URL: <https://nodejs.org/docs> (дата обращения: 03.05.2026).
11. Express.js Guide. — URL: <https://expressjs.com> (дата обращения: 03.05.2026).
12. MySQL Reference Manual. — URL: <https://dev.mysql.com/doc> (дата обращения: 03.05.2026).
13. EJS Documentation. — URL: <https://ejs.co> (дата обращения: 03.05.2026).

14. Visual Studio Code Documentation. — URL:  
<https://code.visualstudio.com/docs> (дата обращения:  
03.05.2026).

15. Гаврилов, М. В. Информатика и информационные технологии  
: учебник / М. В. Гаврилов, В. А. Климов. — 5-е изд. — Москва  
: Юрайт, 2024. — 383 с.

|      |        |      |          |         |      |                       |      |
|------|--------|------|----------|---------|------|-----------------------|------|
|      |        |      |          |         |      | КР-09.02.07-В-63-26ПЗ | Лист |
|      |        |      |          |         |      |                       | 52   |
| Изм. | Кол.уч | Лист | № докум. | Подпись | Дата |                       |      |

# Ег диаграмма базы данных



## Диаграмма последовательности сценария входа пользователя

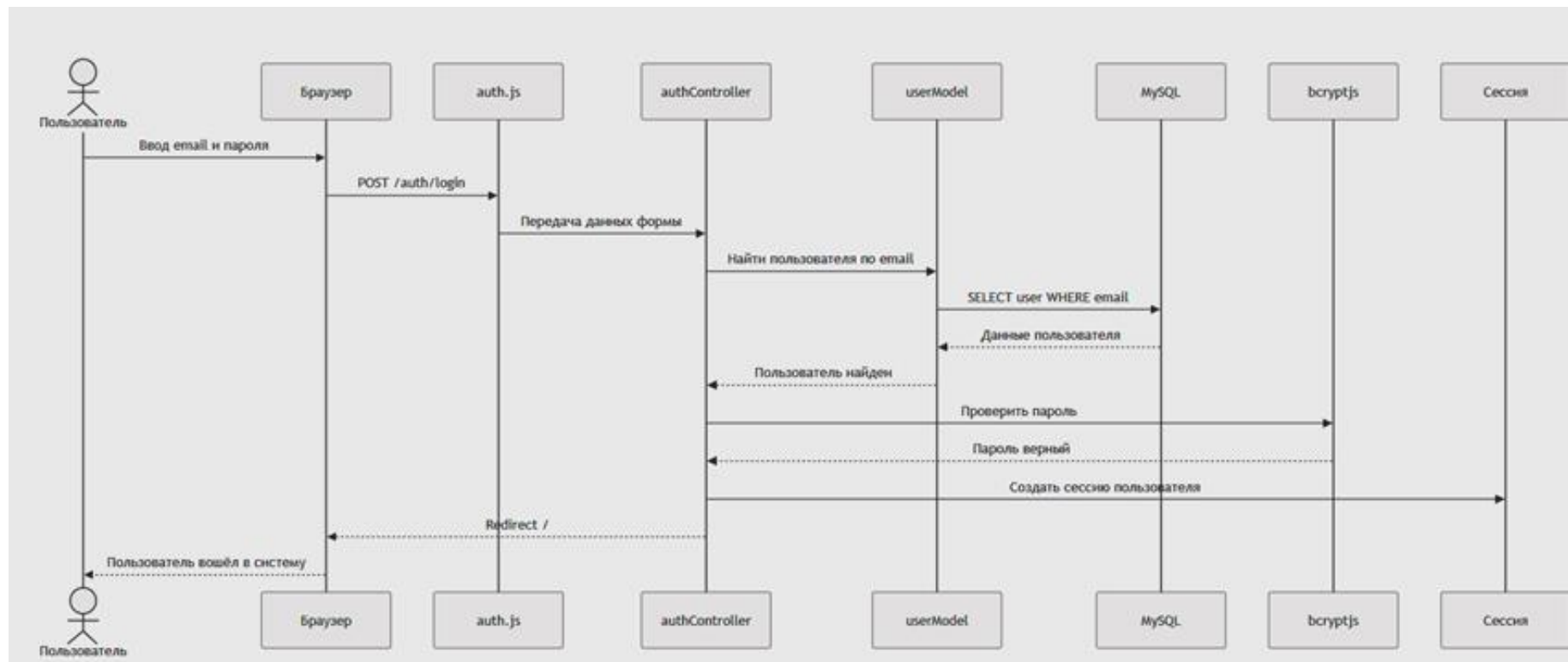
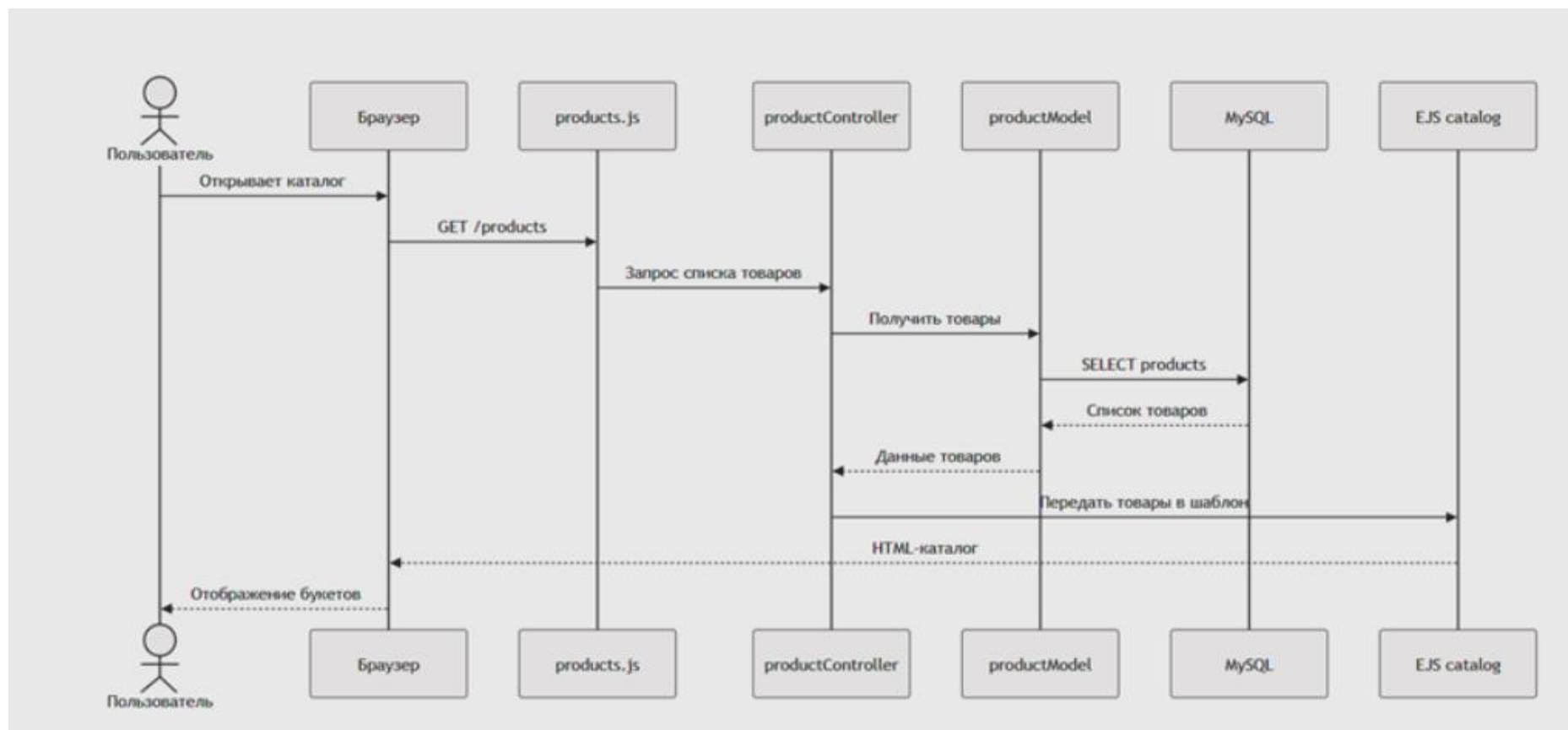
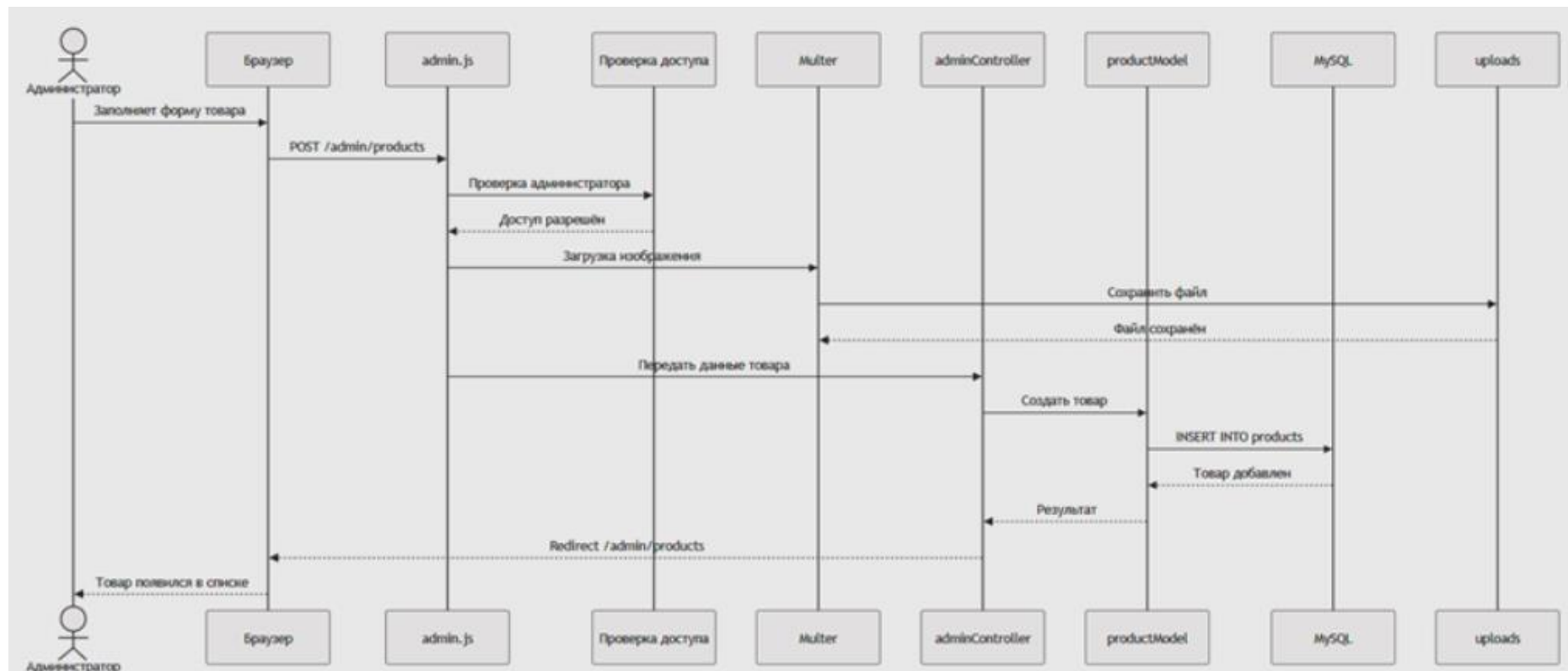


Диаграмма последовательности для просмотра каталога товаров пользователем



## ПРИЛОЖЕНИЕ Г

### Диаграмма последовательности добавления нового товара администратором





## Диаграмма последовательности добавления нового товара администратором

Корзина



Красные розы

2 590 Р

Обновить

Удалить



Белые розы 15 шт

3 290 Р

Обновить

Удалить



Жёлтые тюльпаны 25 шт

2 990 Р

Обновить

Удалить

Сумма: 8 870 Р

Скидка: 0 Р

Итого: 8 870 Р

Для оформления заказа необходимо войти.